

PVL333

System On Chip

Introduction to System Approach

Dr. Robin Singla
Assistant Professor, TIET
Ph.D. IITB

COMPONENTS OF THE SYSTEM

- A system-on-chip (SOC) architecture is an ensemble of processors, memories, and interconnects tailored to an application domain.
- The term architecture denotes the operational structure and the user's view of the system. Over time, it has evolved to include both the functional specification and the hardware implementation.
- The system architecture defines the system-level building blocks, such as processors and memories, and their interconnection.

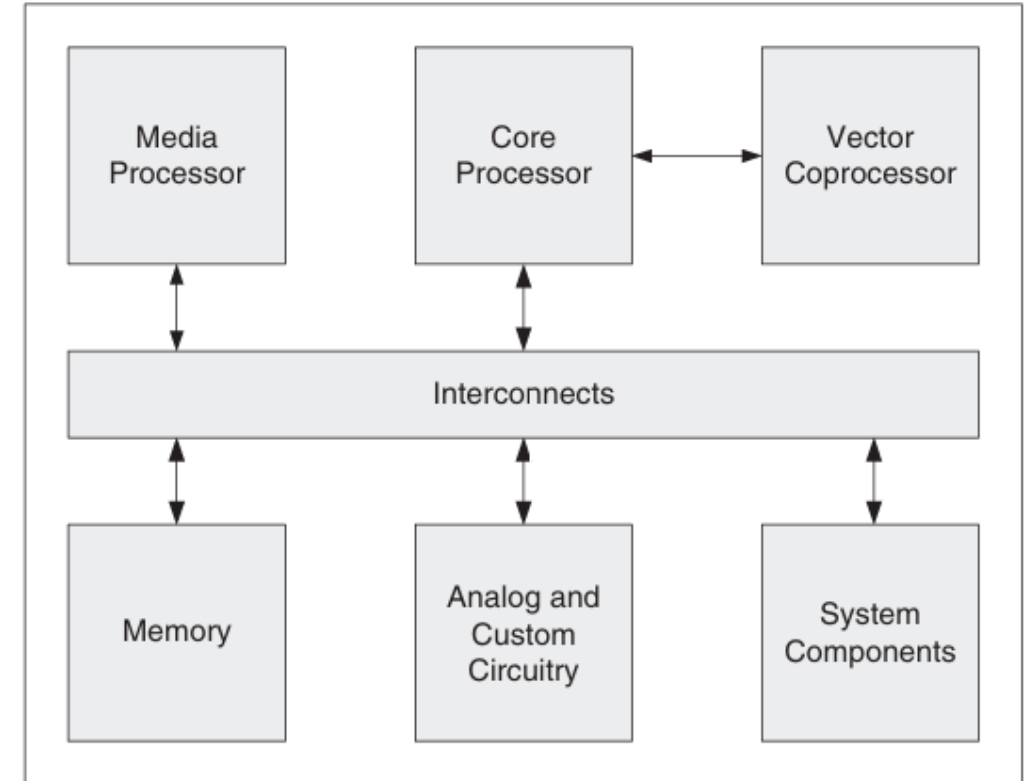


Figure 1.5 A basic SOC system model.

HARDWARE AND SOFTWARE: PROGRAMMABILITY VERSUS PERFORMANCE

- A fundamental decision in SOC design is to choose which components in the system are to be implemented in hardware and in software.

TABLE 1.1 Benefits and Drawbacks of Software and Hardware Implementations

	Benefits	Drawbacks
Hardware	Fast, low power consumption	Inflexible, unadaptable, complex to build and test
Software	Flexible, adaptable, simple to build and test	Slow, high power consumption

HARDWARE AND SOFTWARE: PROGRAMMABILITY VERSUS PERFORMANCE

- A software implementation is usually executed on a general-purpose processor (GPP), which interprets instructions at run time.
- This architecture offers flexibility and adaptability, and provides a way of sharing resources among different applications.
- However, the hardware implementation of the ISA is generally slower and more power hungry than implementing the corresponding function directly in hardware without the overhead of fetching and decoding instructions.
- Most software developers use high-level languages and tools that enhance productivity, such as program development environments, optimising compilers, and performance profilers.
- In contrast, the direct implementation of applications in hardware results in custom application-specific integrated circuits (ASICs), which often provide high performance at the expense of programmability — and hence flexibility, productivity, and cost.

HARDWARE AND SOFTWARE: PROGRAMMABILITY VERSUS PERFORMANCE

- Given that hardware and software have complementary features, many SOC designs aim to combine the individual benefits of the two.
- The obvious method is to implement the performance-critical parts of the application in hardware and the rest in software.
- For instance, if 90% of an application's execution time is spent on 10% of its source code, a 10-fold speedup is achievable if that 10% of the code is efficiently implemented in hardware.
- Custom ASIC hardware and software on GPPs can be seen as two extremes in the technology spectrum, with different trade-offs in programmability and performance; there are various technologies that lie between these two extremes.

HARDWARE AND SOFTWARE: PROGRAMMABILITY VERSUS PERFORMANCE

- The two more well-known ones are application-specific instruction processors (ASIPs) and field-programmable gate arrays (FPGAs).

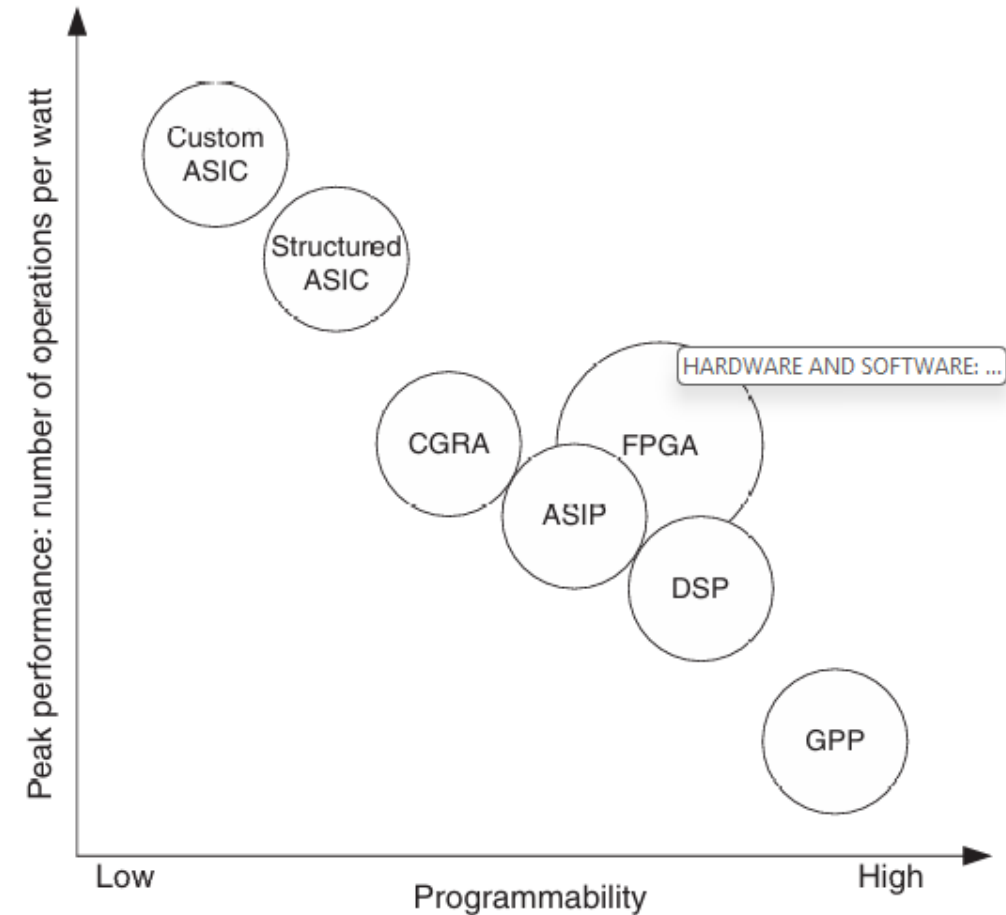


Figure 1.6 A simplified technology comparison: programmability versus performance. GPP, general-purpose processor; CGRA, coarse-grained reconfigurable architecture.

HARDWARE AND SOFTWARE: PROGRAMMABILITY VERSUS PERFORMANCE

- An **ASIP** is a processor with an instruction set customized for a specific application or domain. Custom instructions efficiently implemented in hardware are often integrated into a base processor with a basic instruction set. This capability often improves upon the conventional approach of using standard instruction sets to fulfil the same task while preserving its flexibility.
- An **FPGA** typically contains an array of computation units, memories, and their interconnections, and all three are usually programmable in the field by application builders. FPGA technology often offers a good compromise: It is faster than software while being more flexible and having shorter development times than custom ASIC hardware implementations. Like GPPs, they are offered as off-the-shelf devices that can be programmed without going through chip fabrication. Because of the growing demand to reduce time-to-market and the rising cost of chip fabrication, FPGAs are becoming more popular for implementing digital designs.

PROCESSOR ARCHITECTURES

- Typically, processors are characterised either by their application or by their architecture (or structure).

TABLE 1.2 Processor Examples as Identified by Function

Processor Type	Application
Graphics processing unit (GPU)	3-D graphics; rendering, shading, texture
Digital signal processor (DSP)	Generic, sometimes used with wireless
Media processor	Video and audio signal processing
Network processor	Routing, buffering

TABLE 1.3 Processor Examples as Identified by Architecture

Processor Type	Architecture/Implementation Approach
SIMD	Single instruction applied to multiple functional units (processors)
Vector (VP)	Single instruction applied to multiple pipelined registers
VLIW	Multiple instructions issued each cycle under compiler control
Superscalar	Multiple instructions issued each cycle under hardware control

PROCESSOR ARCHITECTURES

- Exploiting program parallelism is one of the most important goals in computer architecture.
 - **Instruction-level parallelism (ILP)** means that multiple operations can be executed in parallel within a program. ILP may be achieved with hardware, compiler, or operating system techniques.
 - **At the loop level**, consecutive iterations are ideal candidates for parallel execution, provided there is no data dependency between them.
 - Next, there is parallelism available at the **procedure level**, which depends largely on the algorithms used in the program.
 - Finally, multiple independent **programs** can execute in parallel.

PROCESSOR ARCHITECTURES

1. Processor: A Functional View

- Table 1.4 shows different SOC designs and the processor used in each design. For these examples, we can characterise them as general-purpose or special-purpose, with support for gaming or signal-processing applications. This functional view provides little insight into the underlying hardware implementation.

TABLE 1.4 Processor Models for Different SOC Examples

SOC	Application	Base ISA	Processor Description
Freescape e600 [101]	DSP	PowerPC	Superscalar with vector extension
ClearSpeed CSX600 [59]	General	Proprietary ISA	Array processor of 96 processing elements
PlayStation 2 [147, 187, 237]	Gaming	MIPS	Pipelined with two vector coprocessors
ARM VFP11 [23]	General	ARM	Configurable vector coprocessor

PROCESSOR ARCHITECTURES

2. Processor: An Architectural View

- The architectural view of the system describes the actual implementation at least in a broad-brush way. For sophisticated architectural approaches, more detail is needed to fully understand the implementation.
- **Simple Sequential Processor:** Sequential processors directly implement the sequential execution model. These processors process instructions sequentially from the instruction stream. The next instruction is not processed until all execution for the current instruction is complete and its results have been committed.

PROCESSOR ARCHITECTURES

2. Processor: An Architectural View

Simple Sequential Processor

1. fetching the instruction into the instruction register (IF),
2. decoding the opcode of the instruction (ID),
3. generating the address in the memory of any data item residing there (AG),
4. fetching data operands into executable registers (DF),
5. executing the specified operation (EX),
and
6. writing back the result to the register file (WB).

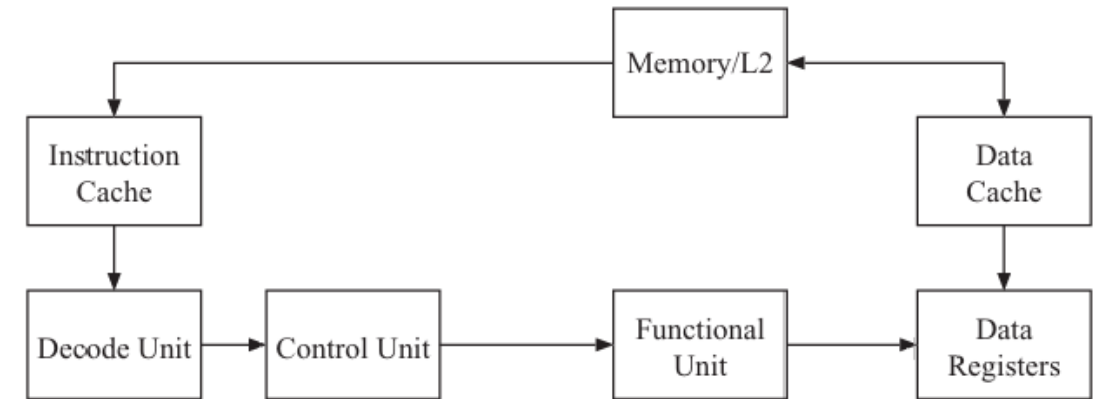


Figure 1.8 Sequential processor model.

PROCESSOR ARCHITECTURES

2. Processor: An Architectural View

Pipelined Processor

- Pipelining is a straightforward approach to exploiting parallelism that is based on concurrently performing different phases (instruction fetch, decode, execution, etc.) of processing an instruction.
- Pipelining assumes that these phases are independent between different operations and can be overlapped — when this condition does not hold, the processor stalls the downstream phases to enforce the dependency.
- Thus, multiple operations can be processed simultaneously, with each operation at a different phase of its processing.

PROCESSOR ARCHITECTURES

2. Processor: An Architectural View

Pipelined Processor

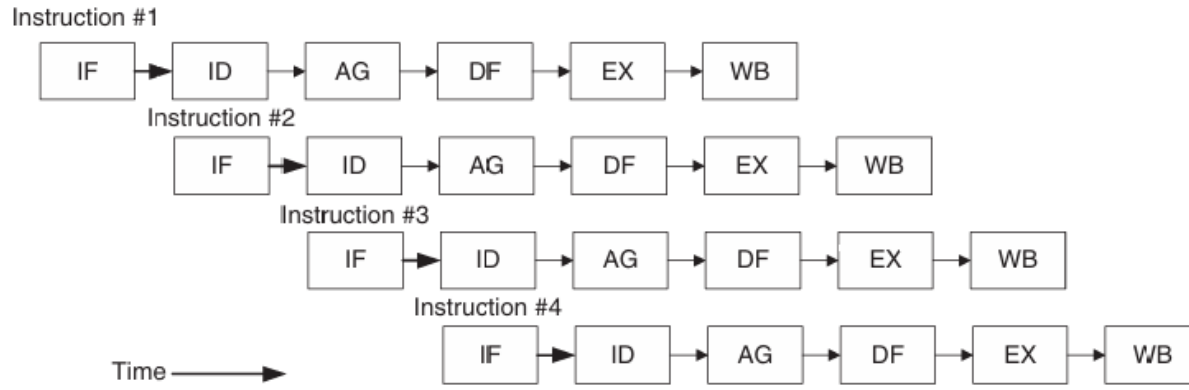


Figure 1.9 Instruction timing in a pipelined processor.

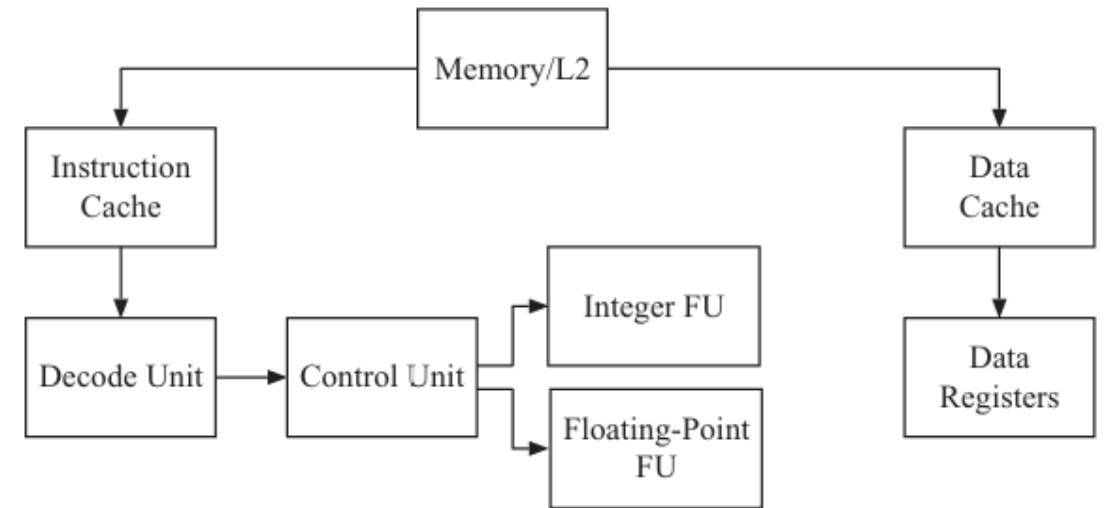


Figure 1.10 Pipelined processor model.

PROCESSOR ARCHITECTURES

2. Processor: An Architectural View

ILP – Instruction Level Parallelism

- While pipelining does not necessarily lead to executing multiple instructions at exactly the same time, there are other techniques that do.
- These techniques may use a combination of static scheduling and dynamic analysis to concurrently perform the evaluation phase of several operations, potentially yielding an execution rate of more than one operation per cycle.

PROCESSOR ARCHITECTURES

2. Processor: An Architectural View

ILP – Instruction Level Parallelism

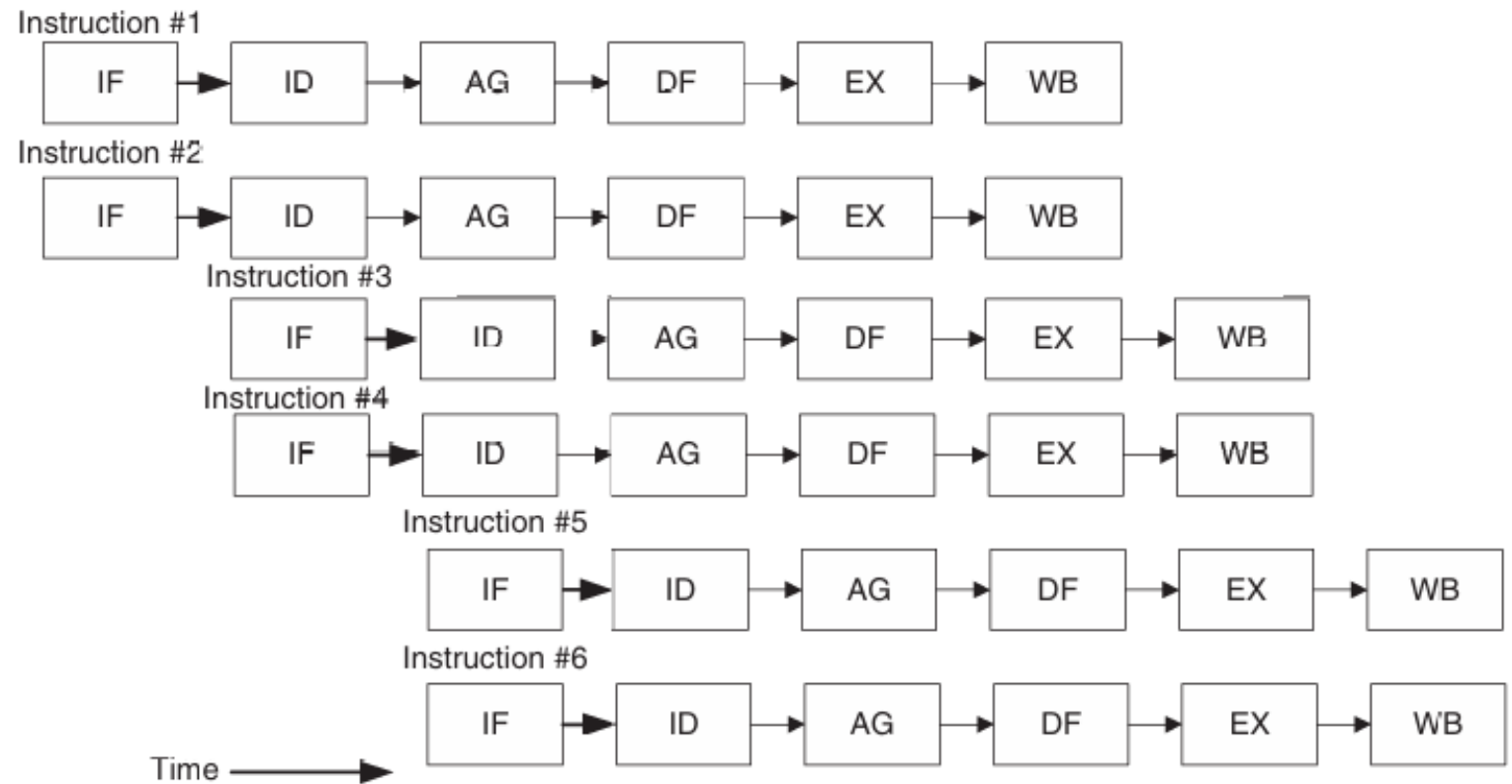


Figure 1.11 Instruction timing in a pipelined ILP processor.

PROCESSOR ARCHITECTURES

2. Processor: An Architectural View

SIMD Architectures: Array and Vector Processors

Single instruction stream, multiple data streams

- The SIMD class of processor architecture includes both array and vector processors.
- The SIMD processor is a natural fit for certain regular data structures, such as vectors and matrices.
- From the perspective of an assembly-level programmer, programming a SIMD architecture appears very similar to programming a simple processor, except that some operations compute on aggregate data.
- Since these regular structures are widely used in scientific programming, SIMD processors have been very successful in these environments.
- The two popular types of SIMD processors are the array processor and the vector processor.

PROCESSOR ARCHITECTURES

2. Processor: An Architectural View

SIMD Architectures: Array and Vector Processors

Single instruction stream, multiple data streams

- An **array processor** consists of many interconnected processor elements, each having their own local memory space.
- A **vector processor** consists of a single processor that references a global memory space and has special function units that operate on vectors.

PROCESSOR ARCHITECTURES

2. Processor: An Architectural View

Multiprocessors

- Multiple processors can cooperatively execute to solve a single problem by using some form of interconnection for sharing results.
- In this configuration, each processor executes completely independently, although most applications require some form of synchronisation during execution to pass information and data between processors.
- Since multiple processors share memory and execute separate program tasks (MIMD [multiple instruction stream, multiple data stream]), their proper implementation is significantly more complex than that of an array processor.
- The interconnection network in the multiprocessor passes data between processor elements and synchronizes the independent execution streams between processor elements.

MEMORY AND ADDRESSING

- SOC applications vary significantly in memory requirements.
- In one case, the memory structure can be as simple as the program residing entirely in an on-chip read-only memory (ROM), with the data in on-chip RAM.
- In another case, the memory system might support an elaborate operating system that requires a large off-chip memory (system-on-a-board), with a memory management unit and a cache hierarchy.
- Why not simply include memory with the processor on the die? This has many attractions:
 1. It improves the accessibility of memory, improving both memory access time and bandwidth.
 2. It reduces the need for a large cache.
 3. It improves performance for memory-intensive applications.

MEMORY AND ADDRESSING

- But there are problems.
- The first problem is that DRAM memory process technology differs from standard microprocessor process technology, and would cause some sacrifice in achievable bit density.
- The second problem is more serious: If memory were restricted to the processor die, its size would be correspondingly limited. Applications that require very large amounts of real memory would be crippled.

MEMORY AND ADDRESSING

- Thus, the conventional processor die model has evolved to implement multiple robust homogeneous processors sharing the higher levels of a two - or three-level cache structure with the main memory off-die, on its own multidie module.
- From a design complexity point of view, this has the advantage of being a “universal ” solution: one implementation fits all applications, although not necessarily equally well.
- So, while a great deal of design effort is required for such an implementation, the production quantities can be large enough to justify the costs.

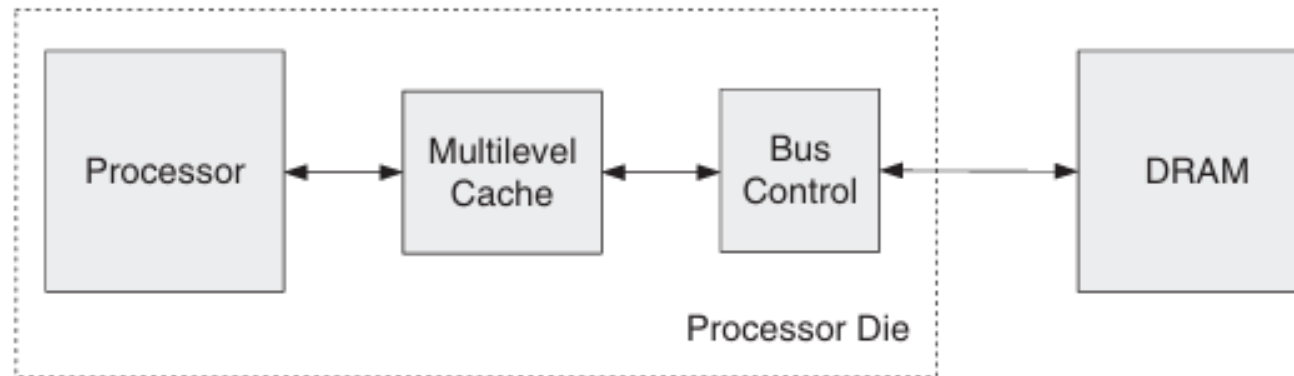


Figure 1.16 Processors with memory off-die.

MEMORY AND ADDRESSING

- An alternative to this approach is clear. For specific applications, whose memory size can be bounded, we can implement an integrated memory SOC.

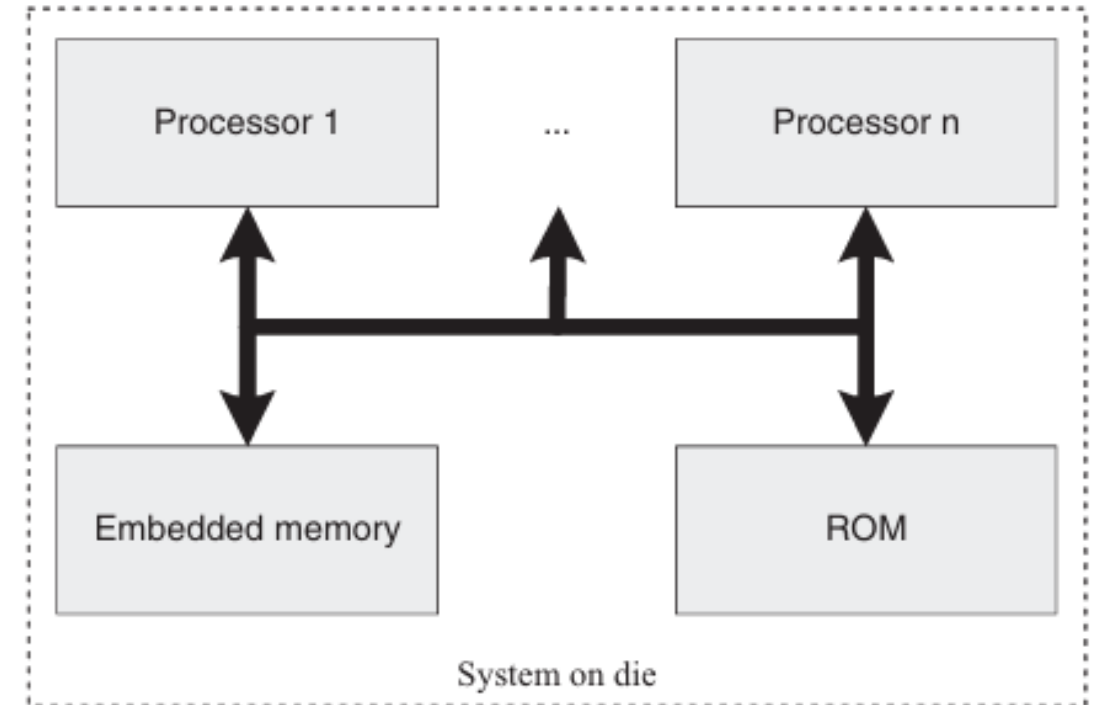


Figure 1.17 System on a chip: processors and memory.

MEMORY AND ADDRESSING

The memory can be centralised or distributed. Even here, the memory can appear to the programmer as a single (centralized) shared memory, even though it is implemented in several distributed modules

TABLE 1.11 SOC Memory Considerations

Issue	Implementation	Comment
Memory placement	On-die Off-die	Limited and fixed size System on a board, slow access, limited bandwidth
Addressing	Real addressing Virtual addressing	Limited size, simple OS Much more complex, require TLB, in-order instruction execution support
Arrangement (as programmed for multiple processors)	Shared memory Message passing	Requires hardware support Additional programming
Arrangement (as implemented)	Centralized Distributed	Limited by chip considerations Can be clustered with a processor or other memory modules

MEMORY AND ADDRESSING

SOC Memory Examples

Table shows several SOC designs and their cache and memory configurations. It is important for SOC designers to consider whether to put RAM and ROM on-die or off-die.

TABLE 1.12 Memory Hierarchy for Different SOC Examples

SOC	Application	Cache Size	On-Die/ Off-Die	Real/ Virtual
NetSilicon NET + 40 [184]	Networking	4-KB I-cache, 4-KB D-cache	Off	Real
NetSilicon NS9775 [185]	Printing	8-KB I-cache, 4-KB D-cache	Off	Virtual
NXP LH7A404 [186]	Networking	16-KB I-cache, 8KB D-Cache	On	Virtual
Motorola RSVP [58]	Multimedia	Tile buffer memory	Off	Real

MEMORY AND ADDRESSING

SOC Memory Examples

Table shows various examples of SOC embedded memory macro cell.

TABLE 1.13 Example SOC Embedded Memory Macro Cell (See Chapter 4 for the Discussion on Cell Types)

Vendor	Cell Type (Typical)	SOC User (Typical)
Virage Logic	6T (SRAM)	SigmaTel/ARM
ATMOS	1T (eDRAM)	Philips
IBM	1T (eDRAM)	IBM

Note: T refers to the number of transistors in a 1-bit cell.

MEMORY AND ADDRESSING

Addressing: The Architecture of Memory

- The user's view of memory primarily consists of the addressing facilities available to the programmer.
- Some of these facilities are available to the application programmer and some to the operating system programmer.
- Virtual memory enables programs requiring larger storage than the physical memory to run and allows separation of address spaces to protect unauthorised access to memory regions when executing multiple application programs.
- When virtual addressing facilities are properly implemented and programmed, memory can be efficiently and securely accessed.

MEMORY AND ADDRESSING

Addressing: The Architecture of Memory

- Virtual memory is often supported by a memory management unit. Conceptually, the physical memory address is determined by a sequence of (at least) three steps:
 1. The application produces a process address . This, together with the process or user ID , defines the virtual address : $\text{virtual address} = \text{offset} + (\text{program}) \text{ base} + \text{index}$, where the offset is specified in the instruction while the base and index values are in specified registers.
 2. Since multiple processes must cooperate in the same memory space, the process addresses must be coordinated and relocated. This is typically done by a segment table. Upper bits of the virtual address are used to address a segment table, which has a (predetermined) base and bound values for the process, resulting in a system address : $\text{system address} = \text{virtual address} + (\text{process}) \text{ base}$, where the system address must be less than the bound .

MEMORY AND ADDRESSING

Addressing: The Architecture of Memory

- Usually, the tables (segment and page) performing address translation are in memory, and a mechanism for the translation called the translation lookaside buffer (TLB) must be used to speed up this translation. A TLB is a simple register system, usually consisting of 64 to 256 entries, that stores recent address translations for reuse.

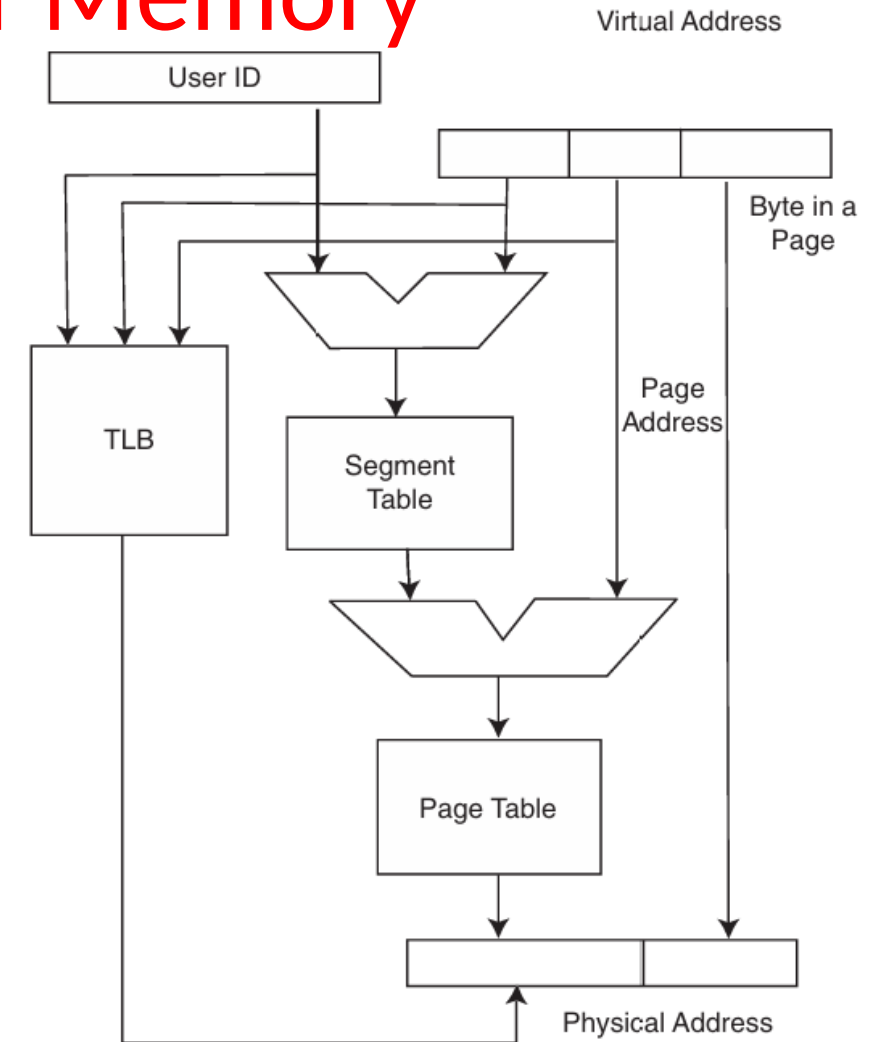


Figure 1.18 Virtual-to-real address mapping with a TLB bypass.

MEMORY AND ADDRESSING

Addressing: The Architecture of Memory

- Virtual memory is often supported by a memory management unit. Conceptually, the physical memory address is determined by a sequence of (at least) three steps:

3. **Virtual versus real.** For many SOC applications (and all generic systems), the memory space exceeds the available (real) implemented memory.

Here, memory is implemented on disk, and only recently used regions (pages) are brought into memory.

The available pages are located by a page table.

The upper bits of the system address access a page table. If the data for this page have been loaded from the disk, the location in memory will be provided as the upper address bits of the “ real ” or physical memory address.

The lower bits of the real address are the same as the corresponding lower bits of the virtual address.

SYSTEM - LEVEL INTERCONNECTION

- SOC technology typically relies on the interconnection of predesigned circuit modules (known as intellectual property [IP] blocks) to form a complete system, which can be integrated onto a single chip.
- In this way, the design task is raised from a circuit level to a system level.
- Central to the system-level performance and the reliability of the finished product is the method of interconnection used.
- A well-designed interconnection scheme should have robust and efficient communication protocols, unambiguously defined as published standards.
- This facilitates interoperability between IP blocks designed by different people from different organisations and encourages design reuse.
- It should enable efficient communication between modules, maximising the degree of parallelism.

SYSTEM - LEVEL INTERCONNECTION

- SOC interconnect methods can be classified into two main approaches: buses and network - on - chip

1. Bus - Based Approach

- With the bus-based approach, IP blocks are designed to conform to published bus standards (such as ARM's Advanced Microcontroller Bus Architecture [AMBA] or IBM's CoreConnect).
- Communication between modules is achieved through the sharing of the physical connections of address, data, and control bus signals.
- This is a common method used for the SOC system-level interconnect. Usually, two or more buses are employed in a system, organised hierarchically.
- To optimise system-level performance and cost, the bus closest to the CPU has the highest bandwidth, and the bus farthest from the CPU has the lowest bandwidth.

SYSTEM - LEVEL INTERCONNECTION

1. Bus - Based Approach

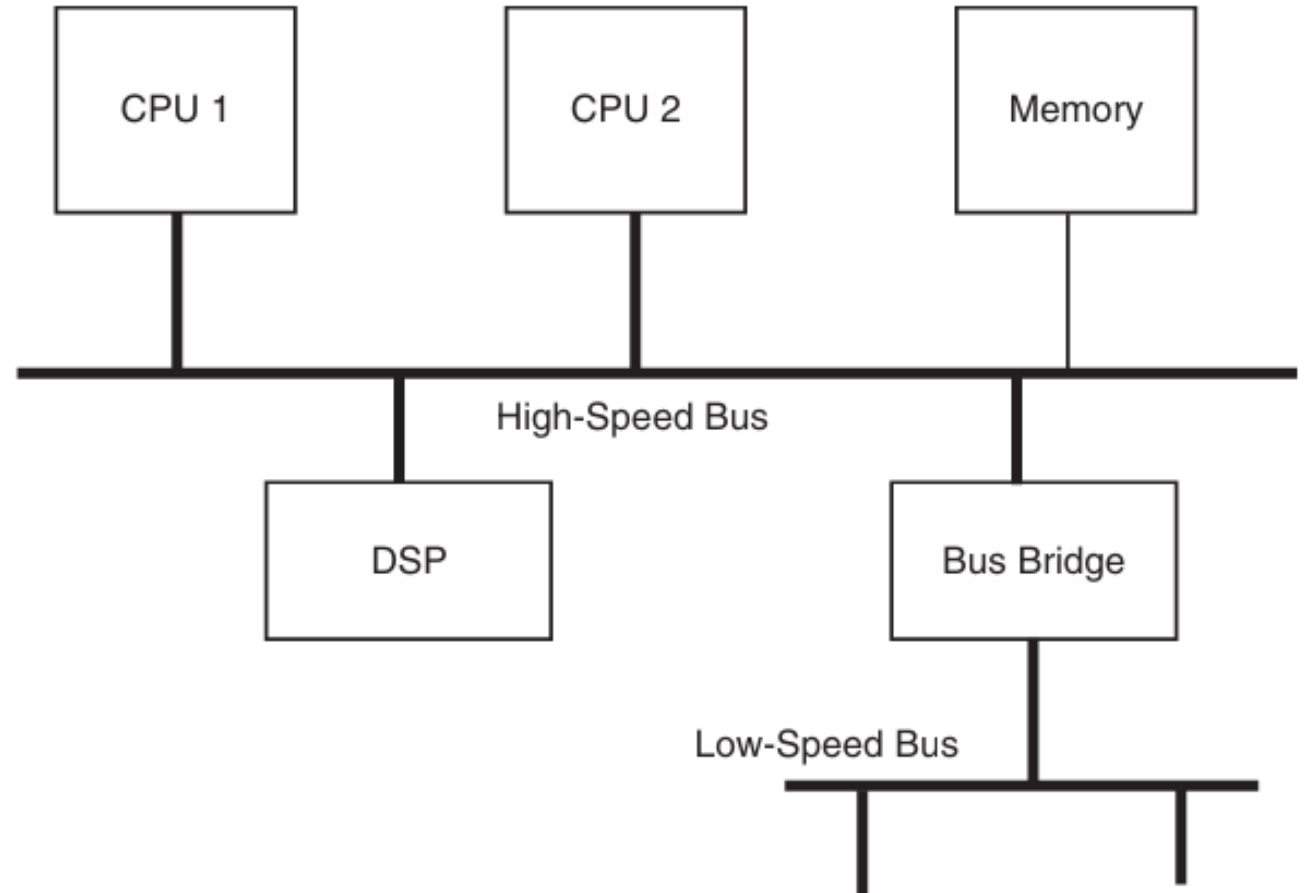


Figure 1.19 SOC system-level interconnection: bus-based approach.

SYSTEM - LEVEL INTERCONNECTION

2. Network - on - Chip Approach

- A network-on-chip system consists of an array of switches, either dynamically switched, as in a crossbar, or statically switched, as in a mesh.
- The crossbar approach uses asynchronous channels to connect synchronous modules that can operate at different clock frequencies.
- This approach has the advantage of higher throughput than a bus-based system while making integration of a system with multiple clock domains easier.

SYSTEM - LEVEL INTERCONNECTION

2. Network - on - Chip Approach

- In a simple statically switched network, each node contains processing logic forming the core and its own routing logic.
- The interconnect scheme is based on a two-dimensional mesh topology.
- All communications between switches are conducted through data packets, routed through the router interface circuit within each node.
- Since the interconnections between switches have a fixed distance, interconnect-related problems such as wire delay and cross-talk noise are much reduced.

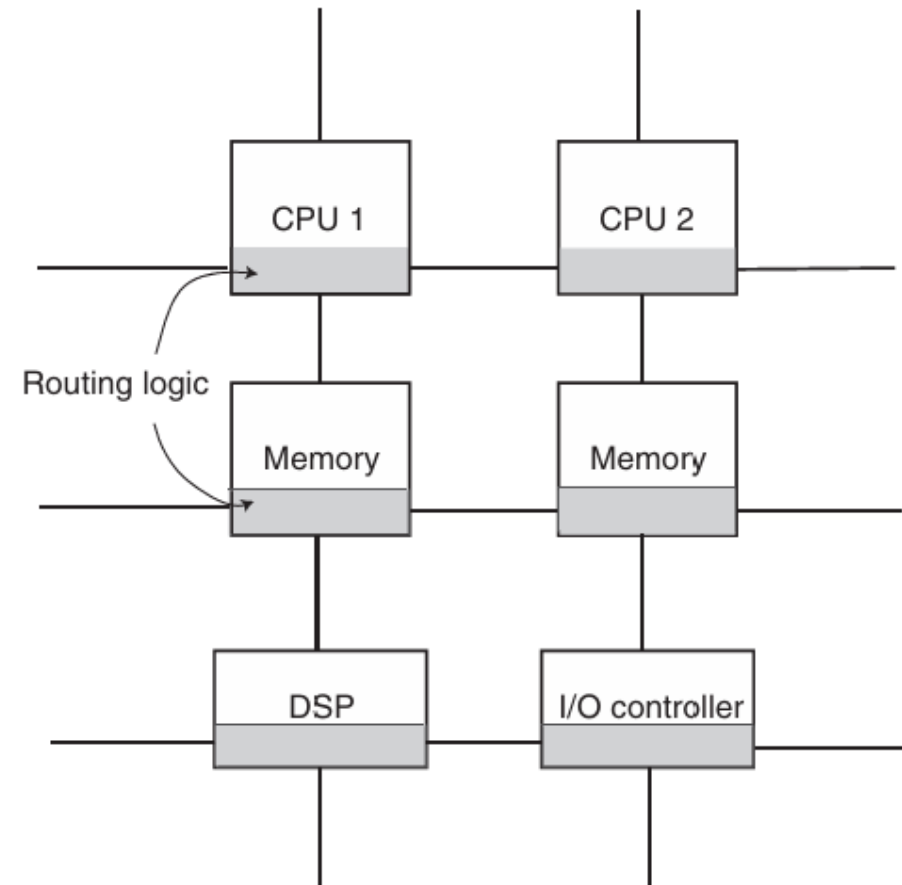


Figure 1.20 SOC system-level interconnection: network-on-chip approach.

AN APPROACH FOR SOC DESIGN

Two important ideas in a design process are figuring out the requirements and specifications, and iterating through different stages of design toward an efficient and effective completion.

1. Requirements and Specifications

- Requirements and specifications are fundamental concepts in any system design situation.
- There must be a thorough understanding of both before a design can begin.
- They are useful at the beginning and at the end of the design process: at the beginning, to clarify what needs to be achieved; and at the end, as a reference against which the completed design can be evaluated.
- The system requirements are largely externally generated criteria for the system.
- They may come from competition, from sales insights, from customer requests, from product profitability analysis, or from a combination.

AN APPROACH FOR SOC DESIGN

1. Requirements and Specifications

- It is important for the designer to analyse the requirements expressions carefully, and to spend sufficient time in understanding the market situation to determine all the factors expressed in the requirements and the priorities those factors imply.
- Some of the factors the designer considers in determining requirements include
 - compatibility with previous designs or published standards,
 - reuse of previous designs,
 - customer requests/complaints,
 - sales reports,
 - cost analysis,
 - competitive equipment analysis, and
 - trouble reports (reliability) of previous products and competitive products.

AN APPROACH FOR SOC DESIGN

2. Design Iteration

- Design is always an iterative process.
- So, the obvious question is how to get the very first, initial design.
- This is the design we can then iterate on and optimise against the design criteria.
- For our purposes, we define several design types based on the stage of the design effort.

AN APPROACH FOR SOC DESIGN

2. Design Iteration

Initial Design

- This is the first design that shows promise in meeting the key requirements, while other performance and cost criteria are not considered.
- For instance, processor, memory or input/output (I/O) should be sized to meet high-priority real-time constraints.
- Promising components and their parameters are selected and analysed to provide an understanding of their expected idealised performance and cost.
- Idealised does not mean ideal; it means a simplified model of the expected area occupied and computational or data bandwidth capability.
- It is usually a simple linear model of performance, such as the expected million instructions per second (MIPS) rate of a processor.

AN APPROACH FOR SOC DESIGN

2. Design Iteration

Optimized Design

- Once the base performance (or area) requirements are met and the base functionality is ensured, then the goal is to minimise the cost (area) and/or the power consumption or the design effort required to complete the design.
- This is the iterative step of the process.
- The first steps of this process use higher-fidelity tools (simulations, trial layouts, etc.) to ensure the initial design actually meets the design specifications and requirements.
- The later steps refine, complete, and improve the design in accordance with the design criteria.

SYSTEM ARCHITECTURE AND COMPLEXITY

- The basic difference between processor architecture and system architecture is that the system adds another layer of complexity, and the complexity of these systems limits the cost savings.
- Historically, the notion of a computer is a single processor plus a memory.
- As long as this notion is fixed (within broad tolerances), implementing that processor on one or more silicon die does not change the design complexity.
- Once die densities enable a scalar processor to fit on a chip, the complexity issue changes.

SYSTEM ARCHITECTURE AND COMPLEXITY

- The result is significantly greater design complexity (see Figure 1.23).
- Instead of the 100,000-transistor processors, our advanced processor has millions of transistors;
- the multilevel caches are also complex, as is the need to coordinate (synchronise) the multiple processors, since they require a consistent image of the contents of memory.

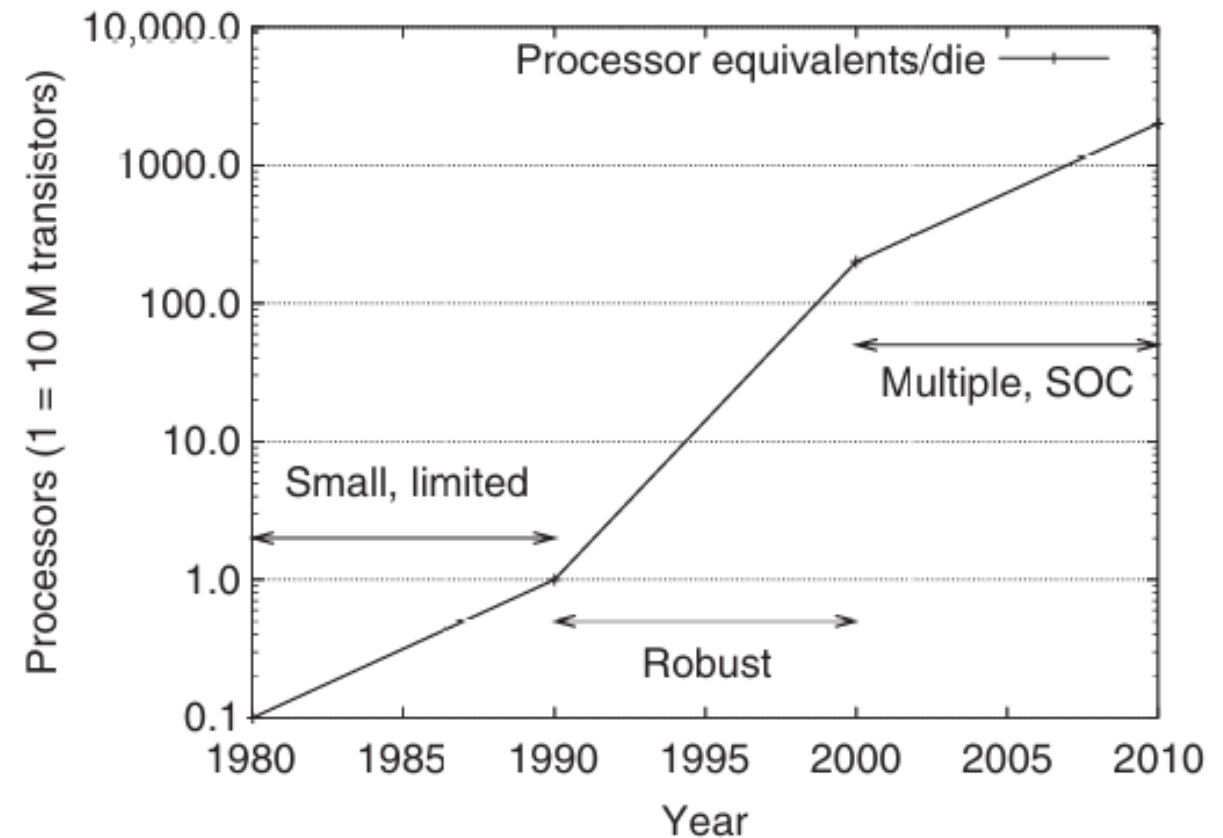


Figure 1.23 Complexity of design.

SYSTEM ARCHITECTURE AND COMPLEXITY

- The obvious way to manage this complexity is to reuse designs.
- So, reusing several simpler processor designs implemented on a die is preferable to a new, more advanced, single processor.
- This is especially true if we can select specific processor designs suited to particular parts of an application.
- For this to work, we also need a robust interconnection mechanism to access the various processors and memory.

DEALING WITH DESIGN COMPLEXITY

- As design cost and complexity increase, there is a basic trade-off between the design optimisation of the physical product and the cost of the design.
- The balance point depends on n , the number of units expected to be produced.
- There are several approaches to the design productivity problem.
- The most basic approaches are purchasing predesigned components and utilizing reconfigurable devices.

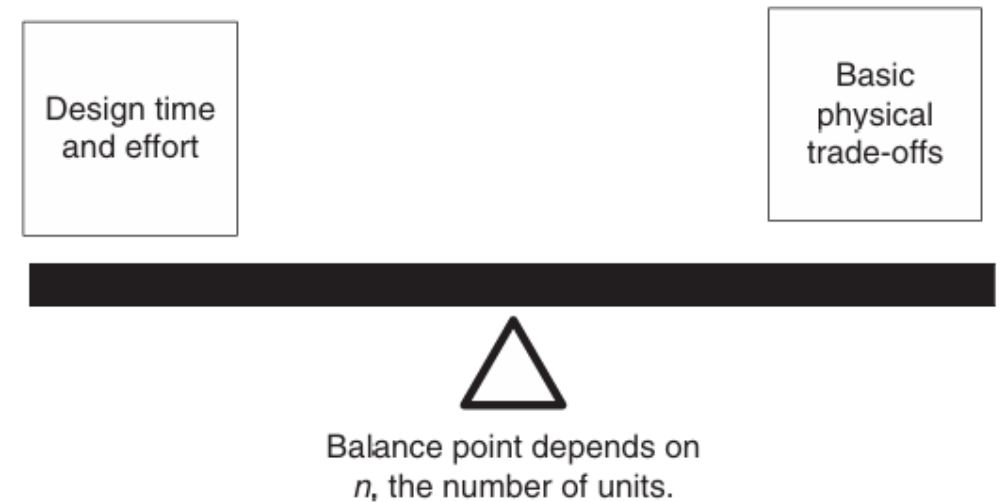


Figure 1.27 The design effort must balance volume.